

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science and Engineering

Fall Semester 2026-27

IACSI204 - Database System Design Lab

Project

TAJ BANQUET HALL BOOKING MANAGEMENT SYSTEM

Student Name: Abishek Palaniappan Ka

Register Number: 25MID0179

Table of Contents

Objectives 3

Scope and assumptions 3

1.1 Functional requirements 4

Non-Functional requirements 4

1.2 ER diagram 5

1.3 Relational Schema 6

1.4 Keys and integrity constraints 7

1.5 Normalization: step-by-step conversion up to 3NF 8

Starting point: Unnormalized form (UNF) 8

First Normal Form (1NF) 9

Second Normal Form (2NF) 9

Third Normal Form (3NF) 10

2.1 Table creation (DDL) 12

2.2 Inserting values into the tables (DML) 13

2.3 Displaying the tables 16

2.4 Update and Delete commands (DML) 18

2.5 Join Queries 18

2.6 Subqueries 19

2.7 Aggregate Functions 20

2.8 Views 21

2.9 Additional useful management queries 22

3.1 Stored Procedure - Create a Booking 24

3.3 Cursor - Display All Booking Details 25

3.4 Trigger - Check Hall Package Capacity 25

3.5 Exception Handling - Demonstration 26

3.6 Testing the Management System 26

Project Overview

This project designs and implements a relational database for Taj Banquet Hall, a venue used for family functions, celebrations and business meetings. The database manages customers, event bookings, hall availability, service packages, payments and optional add-on services. The implementation uses Oracle SQL and PL/SQL.

Objectives

- Store customer and event information consistently.
- Prevent overlapping confirmed bookings for the same hall and date/time.
- Track packages, add-on services, booking amounts and payments.
- Use primary keys, foreign keys, CHECK and UNIQUE constraints to protect data.
- Demonstrate DDL, DML, joins, subqueries, aggregate functions and views.
- Demonstrate PL/SQL procedures, functions, cursors, triggers and exception handling.

Scope and assumptions

The model supports one venue with one bookable hall. Booking start and end times are stored as Oracle DATE values. A booking is treated as active when its status is PENDING or CONFIRMED; CANCELLED bookings do not block a time slot. The sample records are fictional demonstration data. A production system should use transactions and appropriate locking when checking availability.

Phase 1
Requirement Analysis and Database Design

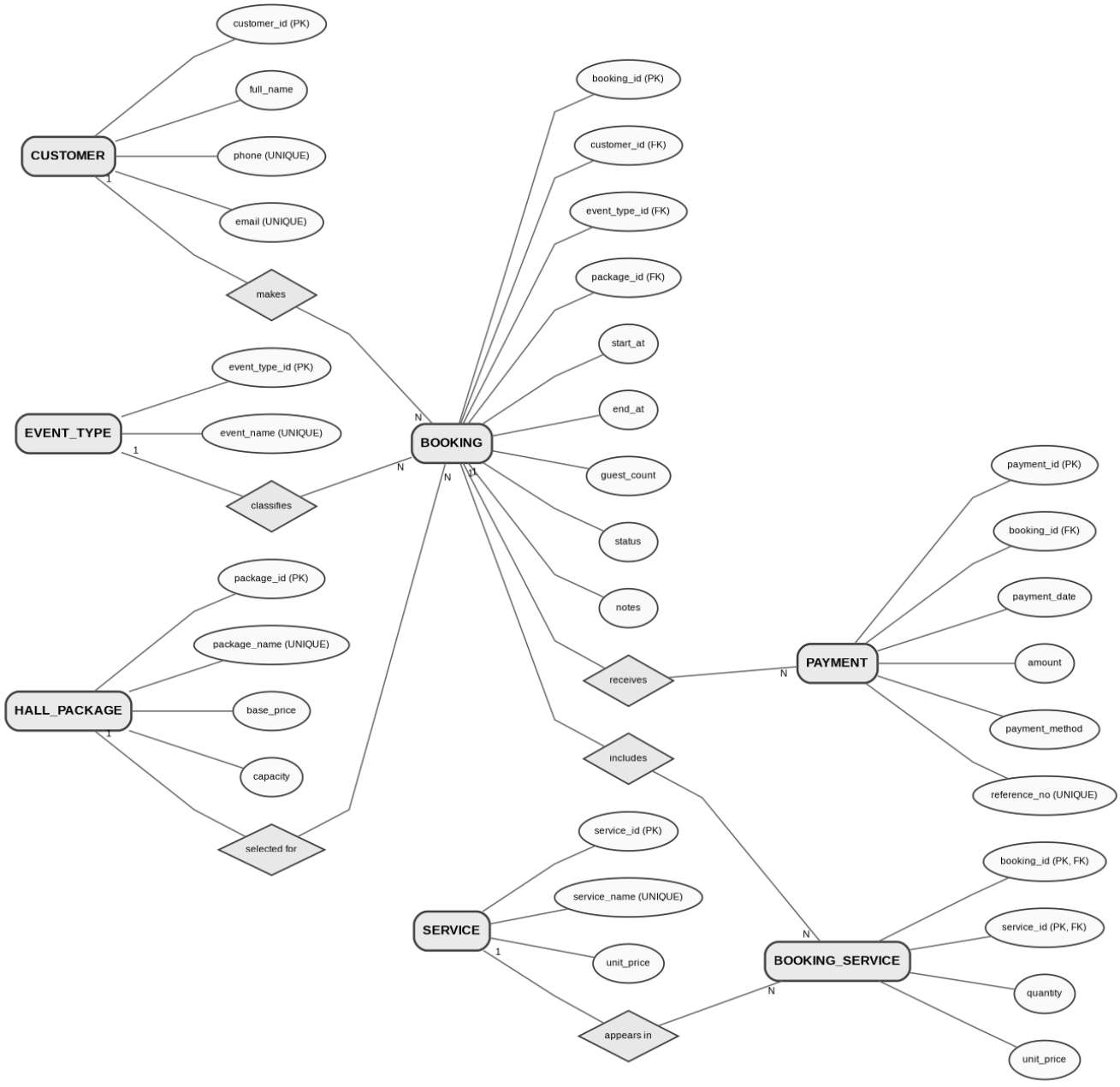
1.1 Functional requirements

1. Register customers with name, phone and email.
2. Maintain event categories and service packages.
3. Create bookings with event date, start time, end time, guest count and status.
4. Attach optional add-on services to bookings and record the quantity and price charged.
5. Record multiple payments against a booking.
6. View booking details, customer booking history, daily schedules, revenue and outstanding balances.
7. Validate that end time is later than start time, guest count is positive, amounts are non-negative and active bookings do not overlap.

Non-Functional requirements

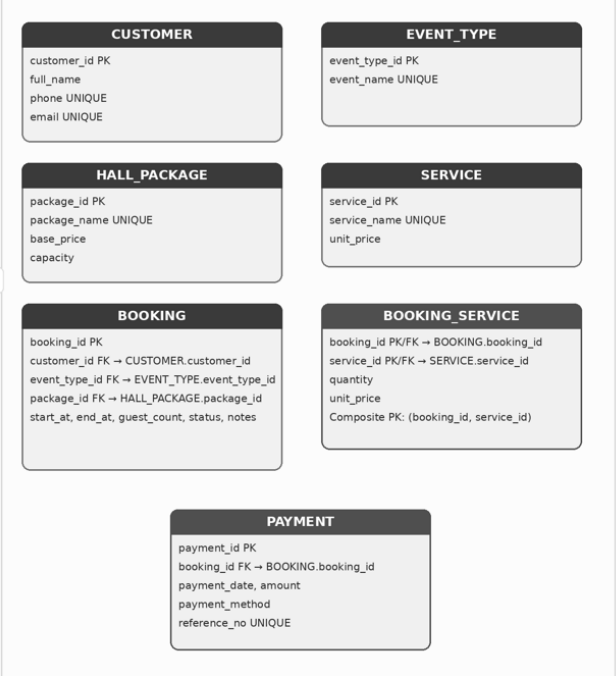
- Data integrity: primary and foreign keys and check constraints.
- Security: grant access only to authorised users in a deployed system.
- Maintainability: separate customers, packages, bookings, services and payments into related tables.
- Reliability: use transactions and handle exceptions rather than silently ignoring errors.
- Performance: index foreign keys and commonly searched booking dates in a production deployment.

1.2 ER diagram



PK = Primary Key | FK = Foreign Key | UNIQUE = Unique constraint | Composite PK = (booking_id, service_id)

1.3 Relational Schema



Relation / table	**Columns and keys**
CUSTOMER	customer_id PK; full_name; phone UNIQUE; email UNIQUE
EVENT_TYPE	event_type_id PK; event_name UNIQUE
HALL_PACKAGE	package_id PK; package_name UNIQUE; base_price; capacity
BOOKING	booking_id PK; customer_id FK → CUSTOMER; event_type_id FK → EVENT_TYPE; package_id FK → HALL_PACKAGE; start_at; end_at; guest_count; status; notes
SERVICE	service_id PK; service_name UNIQUE; unit_price

BOOKING_SERVICE	booking_id PK/FK → BOOKING; service_id PK/FK → SERVICE; quantity; unit_price; composite PK (booking_id, service_id)
PAYMENT	payment_id PK; booking_id FK → BOOKING; payment_date; amount; payment_method; reference_no UNIQUE

1.4 Keys and integrity constraints

Constraint	Where used	Purpose
Primary key	ID column in each table; composite key in BOOKING_SERVICE	Uniquely identifies each row.
Foreign key	BOOKING.customer_id, event_type_id, package_id; BOOKING_SERVICE booking_id/service_id; PAYMENT.booking_id	Prevents references to non-existent parent records.
UNIQUE	CUSTOMER.phone/email, EVENT_TYPE.event_name, HALL_PACKAGE.package_name, SERVICE.service_name, PAYMENT.reference_no	Prevents duplicate values where uniqueness is required.
CHECK	Prices >= 0; capacity > 0; quantity > 0; guest_count > 0; amount > 0; valid status/payment method	Rejects invalid values.

Booking time check	end_at > start_at	Ensures the booking has a positive duration.
Availability trigger	Pending/confirmed booking intervals	Prevents overlapping active bookings in the same hall.

1.5 Normalization: step-by-step conversion up to 3NF

Normalization reduces duplicate data and prevents insertion, update and deletion anomalies. The following worked example uses a booking with optional services and payments. In this illustration, one booking may contain several services and several payments.

Starting point: Unnormalized form (UNF)

A poorly designed record might place customer, booking, service and payment details in one record, with repeated columns for Service1/Service2 and Payment1/Payment2. Repeating groups make it difficult to add an arbitrary number of services or payments.

Booking_ID	**Customer_Name**	**Customer_Phone**	**Package**	**Services (repeating group)**	**Payments (repeating group)**
101	Arun Kumar	9000000001	Celebration Package	Projector; Snacks × 100	UPI ₹5,000; Cash ₹2,000
102	Priya S	9000000002	Basic Hall	Projector	UPI ₹3,000

Problems: repeating groups, duplicated customer/package data, difficult updates, and a fixed limit on the number of services/payments.

First Normal Form (1NF)

Rule: each cell contains a single atomic value; there are no repeating groups. Split repeated services and payments into separate rows. The following is an illustrative flattened relation; because one booking can have several services and payments, joining both sets in one table can multiply rows.

Booking_ID	Customer_Name	Customer_Phone	Package_Name	Service_Name	Service_Qty	Payment_Ref	Payment_Amount
101	Arun Kumar	9000000001	Celebration Package	Projector	1	UPI-01	5000
101	Arun Kumar	9000000001	Celebration Package	Snacks	100	CASH-01	2000
102	Priya S	9000000002	Basic Hall	Projector	1	UPI-02	3000

This illustrates atomic values, but customer and package details repeat. A flattened 1NF table is not the final design.

Second Normal Form (2NF)

Rule: be in 1NF and remove partial dependencies on only part of a composite key. In the service line relation, (Booking_ID, Service_ID) identifies a selected service line. Quantity and agreed unit price depend on both values. Customer name depends only on Booking_ID (through the booking's customer), while service name depends only on Service_ID, so those attributes should not stay in the service-line relation.

Relation after decomposition	Attributes

BOOKING_2NF	Booking_ID PK; Customer_ID; Customer_Name; Customer_Phone; Package_ID; Package_Name; Start_At; End_At
BOOKING_SERVICE_2NF	Booking_ID PK/FK; Service_ID PK/FK; Service_Name; Quantity; Agreed_Unit_Price
PAYMENT	Payment_ID PK; Booking_ID FK; Payment_Date; Amount; Payment_Method; Reference_No

The service line now has a composite key, but Service_Name still depends only on Service_ID. Customer_Name/Phone and Package_Name are also dependent on their own entity identifiers rather than directly on Booking_ID.

Third Normal Form (3NF)

Rule: be in 2NF and remove transitive dependencies; non-key attributes should depend on the key, the whole key and nothing but the key. Move customer contact details, package details, event names and service names to their own tables.

Final 3NF relation	**Attributes and keys**
CUSTOMER	customer_id PK; full_name; phone UNIQUE; email UNIQUE
EVENT_TYPE	event_type_id PK; event_name UNIQUE
HALL_PACKAGE	package_id PK; package_name UNIQUE; base_price; capacity
BOOKING	booking_id PK; customer_id FK; event_type_id FK; package_id FK;

	start_at; end_at; guest_count; status; notes
SERVICE	service_id PK; service_name UNIQUE; unit_price
BOOKING_SERVICE	booking_id PK/FK; service_id PK/FK; quantity; unit_price
PAYMENT	payment_id PK; booking_id FK; payment_date; amount; payment_method; reference_no UNIQUE

Why this is 3NF: customer details are stored once in CUSTOMER; event names once in EVENT_TYPE; package attributes once in HALL_PACKAGE; service names/default prices once in SERVICE; and payments are independent rows in PAYMENT. BOOKING_SERVICE.unit_price is intentionally retained because it represents the price agreed at booking time, not merely the current default service price.

Phase 2
SQL IMPLEMENTATION

2.1 Table creation (DDL)

```
SQL> CREATE TABLE customer (
  2    customer_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  3    full_name VARCHAR2(100) NOT NULL,
  4    phone VARCHAR2(15) NOT NULL UNIQUE,
  5    email VARCHAR2(120) UNIQUE
  6  );

Table created.

SQL> CREATE TABLE event_type (
  2    event_type_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  3    event_name VARCHAR2(50) NOT NULL UNIQUE
  4  );

Table created.

SQL> CREATE TABLE hall_package (
  2    package_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  3    package_name VARCHAR2(60) NOT NULL UNIQUE,
  4    base_price NUMBER(10,2) NOT NULL CHECK (base_price >= 0),
  5    capacity NUMBER(5) NOT NULL CHECK (capacity > 0)
  6  );

Table created.

SQL> CREATE TABLE booking (
  2    booking_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  3    customer_id NUMBER NOT NULL REFERENCES customer(customer_id),
  4    event_type_id NUMBER NOT NULL REFERENCES event_type(event_type_id),
  5    package_id NUMBER NOT NULL REFERENCES hall_package(package_id),
  6    start_at DATE NOT NULL,
  7    end_at DATE NOT NULL,
  8    guest_count NUMBER(5) NOT NULL CHECK (guest_count > 0),
  9    status VARCHAR2(12) DEFAULT 'PENDING' NOT NULL
 10      CHECK (status IN ('PENDING','CONFIRMED','CANCELLED','COMPLETED')),
 11    notes VARCHAR2(500),
 12    CONSTRAINT booking_time_ck CHECK (end_at > start_at)
 13  );

Table created.
```

```
SQL> CREATE TABLE service (
  2    service_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  3    service_name VARCHAR2(80) NOT NULL UNIQUE,
  4    unit_price NUMBER(10,2) NOT NULL CHECK (unit_price >= 0)
  5  );

Table created.

SQL> CREATE TABLE booking_service (
  2    booking_id NUMBER NOT NULL REFERENCES booking(booking_id) ON DELETE C
ASCADE,
  3    service_id NUMBER NOT NULL REFERENCES service(service_id),
  4    quantity NUMBER(5) DEFAULT 1 NOT NULL CHECK (quantity > 0),
  5    unit_price NUMBER(10,2) NOT NULL CHECK (unit_price >= 0),
  6    CONSTRAINT booking_service_pk PRIMARY KEY (booking_id, service_id)
  7  );

Table created.

SQL> CREATE TABLE payment (
  2    payment_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  3    booking_id NUMBER NOT NULL REFERENCES booking(booking_id),
  4    payment_date DATE DEFAULT SYSDATE NOT NULL,
  5    amount NUMBER(10,2) NOT NULL CHECK (amount > 0),
  6    payment_method VARCHAR2(20) NOT NULL
  7      CHECK (payment_method IN ('CASH','UPI','CARD','BANK_TRANSFER')),
  8    reference_no VARCHAR2(80) UNIQUE
  9  );

Table created.
```

2.2 Inserting values into the tables (DML)

```
SQL> INSERT INTO customer(full_name,phone,email)VALUES('Arun Kumar','9876543210','arun@gmail.com');

1 row created.

SQL> INSERT INTO customer(full_name,phone,email)VALUES('Priya Sharma','9876543211','priya@gmail.com');

1 row created.

SQL> INSERT INTO customer(full_name,phone,email)VALUES('Rahul Raj','9876543212','rahul@gmail.com');

1 row created.

SQL> INSERT INTO customer(full_name,phone,email)VALUES('Divya S','9876543213','divya@gmail.com');

1 row created.

SQL> INSERT INTO customer(full_name,phone,email)VALUES('Karthik M','9876543214','karthik@gmail.com');

1 row created.
```

```
SQL> INSERT INTO event_type(event_name)VALUES('Wedding');

1 row created.

SQL> INSERT INTO event_type(event_name)VALUES('Birthday');

1 row created.

SQL> INSERT INTO event_type(event_name)VALUES('Reception');

1 row created.

SQL> INSERT INTO event_type(event_name)VALUES('Corporate Event');

1 row created.

SQL> INSERT INTO event_type(event_name)VALUES('Engagement');

1 row created.
```

```
SQL> INSERT INTO hall_package(package_name,base_price,capacity)VALUES('Silver Package',15000,100);

1 row created.

SQL> INSERT INTO hall_package(package_name,base_price,capacity)VALUES('Gold Package',30000,250);

1 row created.

SQL> INSERT INTO hall_package(package_name,base_price,capacity)VALUES('Platinum Package',50000,500);

1 row created.

SQL> INSERT INTO hall_package(package_name,base_price,capacity)VALUES('Diamond Package',75000,750);

1 row created.

SQL> INSERT INTO hall_package(package_name,base_price,capacity)VALUES('Premium Package',100000,1000);

1 row created.
```

```
SQL> INSERT INTO booking(customer_id,event_type_id,package_id,start_at,end_at,guest_count,status,notes)VALUES(1,1,2,TO_DATE('2026-11-10 09:00','YYYY-MM-DD H
H24:MI'),TO_DATE('2026-11-10 18:00','YYYY-MM-DD HH24:MI'),200,'CONFIRMED','Wedding event');

1 row created.

SQL> INSERT INTO booking(customer_id,event_type_id,package_id,start_at,end_at,guest_count,status,notes)VALUES(2,2,1,TO_DATE('2026-11-12 10:00','YYYY-MM-DD H
H24:MI'),TO_DATE('2026-11-12 16:00','YYYY-MM-DD HH24:MI'),80,'PENDING','Birthday party');

1 row created.

SQL> INSERT INTO booking(customer_id,event_type_id,package_id,start_at,end_at,guest_count,status,notes)VALUES(3,3,3,TO_DATE('2026-11-15 08:00','YYYY-MM-DD H
H24:MI'),TO_DATE('2026-11-15 20:00','YYYY-MM-DD HH24:MI'),400,'CONFIRMED','Reception');

1 row created.

SQL> INSERT INTO booking(customer_id,event_type_id,package_id,start_at,end_at,guest_count,status,notes)VALUES(4,4,2,TO_DATE('2026-11-18 09:00','YYYY-MM-DD H
H24:MI'),TO_DATE('2026-11-18 17:00','YYYY-MM-DD HH24:MI'),150,'COMPLETED','Annual corporate meeting');

1 row created.

SQL> INSERT INTO booking(customer_id,event_type_id,package_id,start_at,end_at,guest_count,status,notes)VALUES(5,5,1,TO_DATE('2026-11-20 10:00','YYYY-MM-DD H
H24:MI'),TO_DATE('2026-11-20 15:00','YYYY-MM-DD HH24:MI'),60,'CANCELLED','Engagement ceremony');

1 row created.
```

```
SQL> INSERT INTO service(service_name,unit_price)VALUES('Decoration',5000);
1 row created.
SQL> INSERT INTO service(service_name,unit_price)VALUES('Catering',250);
1 row created.
SQL> INSERT INTO service(service_name,unit_price)VALUES('Photography',8000);
1 row created.
SQL> INSERT INTO service(service_name,unit_price)VALUES('Sound System',3000);
1 row created.
SQL> INSERT INTO service(service_name,unit_price)VALUES('Lighting',4000);
1 row created.
```

```
SQL> INSERT INTO booking_service(booking_id,service_id,quantity,unit_price)VALUES(1,1,1,5000);
1 row created.
SQL> INSERT INTO booking_service(booking_id,service_id,quantity,unit_price)VALUES(1,2,200,250);
1 row created.
SQL> INSERT INTO booking_service(booking_id,service_id,quantity,unit_price)VALUES(1,3,1,8000);
1 row created.
SQL> INSERT INTO booking_service(booking_id,service_id,quantity,unit_price)VALUES(2,1,1,5000);
1 row created.
SQL> INSERT INTO booking_service(booking_id,service_id,quantity,unit_price)VALUES(2,4,1,3000);
1 row created.
SQL> INSERT INTO booking_service(booking_id,service_id,quantity,unit_price)VALUES(3,2,400,250);
1 row created.
SQL> INSERT INTO booking_service(booking_id,service_id,quantity,unit_price)VALUES(3,3,1,8000);
1 row created.
SQL> INSERT INTO booking_service(booking_id,service_id,quantity,unit_price)VALUES(4,4,1,3000);
1 row created.
SQL> INSERT INTO booking_service(booking_id,service_id,quantity,unit_price)VALUES(4,5,1,4000);
1 row created.
SQL> INSERT INTO booking_service(booking_id,service_id,quantity,unit_price)VALUES(5,1,1,5000);
1 row created.
```

```
SQL> INSERT INTO payment(booking_id,payment_date,amount,payment_method,reference_no)VALUES(1,TO_DATE('2026-10-20','YYYY-MM-DD'),30000,'UPI','PAY001');

1 row created.

SQL> INSERT INTO payment(booking_id,payment_date,amount,payment_method,reference_no)VALUES(1,TO_DATE('2026-10-21','YYYY-MM-DD'),15000,'CARD','PAY002');

1 row created.

SQL> INSERT INTO payment(booking_id,payment_date,amount,payment_method,reference_no)VALUES(2,TO_DATE('2026-10-20','YYYY-MM-DD'),5000,'CASH','PAY003');

1 row created.

SQL> INSERT INTO payment(booking_id,payment_date,amount,payment_method,reference_no)VALUES(3,TO_DATE('2026-10-20','YYYY-MM-DD'),40000,'BANK_TRANSFER','PAY00
4');

1 row created.

SQL> INSERT INTO payment(booking_id,payment_date,amount,payment_method,reference_no)VALUES(4,TO_DATE('2026-10-20','YYYY-MM-DD'),10000,'UPI','PAY005');

1 row created.

SQL> COMMIT;

Commit complete.
```

2.3 Displaying the tables

```
SQL> set linesize 200
SQL> set pagesize 100
SQL> set wrap off
SQL> set tab off
SQL> SELECT * FROM customer;

CUSTOMER_ID FULL_NAME                                                                                                PHONE          EMAIL
----------- ---------------------------------------------------------------------------------------------------- -------------- --------------------
----------------------------------------
          1 Arun Kumar                                                                                           9876543210     arun@gmail.com
          2 Priya Sharma                                                                                         9876543211     priya@gmail.com
          3 Rahul Raj                                                                                            9876543212     rahul@gmail.com
          4 Divya S                                                                                              9876543213     divya@gmail.com
          5 Karthik M                                                                                            9876543214     karthik@gmail.com
```

```
SQL> SELECT * FROM event_type;

EVENT_TYPE_ID EVENT_NAME
------------- --------------------------------------------------
            1 Wedding
            2 Birthday
            3 Reception
            4 Corporate Event
            5 Engagement
```

```
SQL> SELECT * FROM hall_package;

PACKAGE_ID PACKAGE_NAME                                                  BASE_PRICE   CAPACITY
---------- ---------------------------------------------------------- ---------- ----------
         1 Silver Package                                                  15000        100
         2 Gold Package                                                    30000        250
         3 Platinum Package                                                50000        500
         4 Diamond Package                                                 75000        750
         5 Premium Package                                                100000       1000
```

```
SQL> SELECT * FROM booking;

BOOKING_ID CUSTOMER_ID EVENT_TYPE_ID PACKAGE_ID START_AT  END_AT    GUEST_COUNT STATUS       NOTES
---------- ----------- ------------- ---------- --------- --------- ----------- ------------ ------------------------------
-----------------------------------------
         1           1             1          2 10-NOV-26 10-NOV-26         200 CONFIRMED    Wedding event
         2           2             2          1 12-NOV-26 12-NOV-26          80 PENDING      Birthday party
         3           3             3          3 15-NOV-26 15-NOV-26         400 CONFIRMED    Reception
         4           4             4          2 18-NOV-26 18-NOV-26         150 COMPLETED    Annual corporate meeting
         5           5             5          1 20-NOV-26 20-NOV-26          60 CANCELLED    Engagement ceremony
```

```
SQL> SELECT * FROM service;

SERVICE_ID SERVICE_NAME                                                                  UNIT_PRICE
---------- ------------------------------------------------------------------------------ ----------
         1 Decoration                                                                           5000
         2 Catering                                                                              250
         3 Photography                                                                          8000
         4 Sound System                                                                         3000
         5 Lighting                                                                             4000
```

```
SQL> SELECT * FROM booking_service;

BOOKING_ID SERVICE_ID   QUANTITY UNIT_PRICE
---------- ---------- ---------- ----------
         1          1          1       5000
         1          2        200        250
         1          3          1       8000
         2          1          1       5000
         2          4          1       3000
         3          2        400        250
         3          3          1       8000
         4          4          1       3000
         4          5          1       4000
         5          1          1       5000
```

```
SQL> SELECT * FROM payment;

PAYMENT_ID BOOKING_ID PAYMENT_D     AMOUNT PAYMENT_METHOD       REFERENCE_NO
---------- ---------- --------- ---------- -------------------- ----------------
         1          1 20-OCT-26      30000 UPI                  PAY001
         2          1 21-OCT-26      15000 CARD                 PAY002
         3          2 20-OCT-26       5000 CASH                 PAY003
         4          3 20-OCT-26      40000 BANK_TRANSFER        PAY004
         5          4 20-OCT-26      10000 UPI                  PAY005
```

2.4 Update and Delete commands (DML)

```
SQL> UPDATE booking SET status='CONFIRMED' WHERE booking_id=2;

1 row updated.

SQL> SELECT * FROM booking;

BOOKING_ID CUSTOMER_ID EVENT_TYPE_ID PACKAGE_ID START_AT  END_AT    GUEST_COUNT STATUS       NOTES
---------- ----------- ------------- ---------- --------- --------- ----------- ------------ --------------------------
-------------------------------------------
         1           1             1          2 10-NOV-26 10-NOV-26         200 CONFIRMED    Wedding event
         2           2             2          1 12-NOV-26 12-NOV-26          80 CONFIRMED    Birthday party
         3           3             3          3 15-NOV-26 15-NOV-26         400 CONFIRMED    Reception
         4           4             4          2 18-NOV-26 18-NOV-26         150 COMPLETED    Annual corporate meeting
         5           5             5          1 20-NOV-26 20-NOV-26          60 CANCELLED    Engagement ceremony

SQL> DELETE FROM payment WHERE payment_id=5;

1 row deleted.

SQL> SELECT * FROM payment;

PAYMENT_ID BOOKING_ID PAYMENT_D     AMOUNT PAYMENT_METHOD       REFERENCE_NO
---------- ---------- --------- ---------- -------------------- ------------------------------------------------------------
         1          1 20-OCT-26      30000 UPI                  PAY001
         2          1 21-OCT-26      15000 CARD                 PAY002
         3          2 20-OCT-26       5000 CASH                 PAY003
         4          3 20-OCT-26      40000 BANK_TRANSFER        PAY004
```

2.5 Join Queries

1. INNER JOIN - Display booking details with customer and event names

```
SQL> SELECT b.booking_id,c.full_name,e.event_name,b.start_at,b.status
  2  FROM booking b JOIN customer c ON b.customer_id=c.customer_id
  3  JOIN event_type e ON b.event_type_id=e.event_type_id;

BOOKING_ID FULL_NAME                                                                                                EVENT_NAME
       START_AT  STATUS
---------- ------------------------------------------------------------------------------------------------------------ ----------
------ --------- ------------
         1 Arun Kumar                                                                                               Wedding
       10-NOV-26 CONFIRMED
         2 Priya Sharma                                                                                             Birthday
       12-NOV-26 CONFIRMED
         3 Rahul Raj                                                                                                Reception
       15-NOV-26 CONFIRMED
         4 Divya S                                                                                                  Corporate Event
       18-NOV-26 COMPLETED
         5 Karthik M                                                                                                Engagement
       20-NOV-26 CANCELLED
```

2. Display booking details with hall package

```
SQL> SELECT b.booking_id,c.full_name,h.package_name,h.base_price
  2  FROM booking b JOIN customer c ON b.customer_id=c.customer_id
  3  JOIN hall_package h ON b.package_id=h.package_id;

BOOKING_ID FULL_NAME                                                                                                PACKAGE_NAME
                BASE_PRICE
---------- ------------------------------------------------------------------------------------------------------------ ----------
---------- ----------
         2 Priya Sharma                                                                                             Silver Package
                    15000
         5 Karthik M                                                                                                Silver Package
                    15000
         1 Arun Kumar                                                                                               Gold Package
                    30000
         4 Divya S                                                                                                  Gold Package
                    30000
         3 Rahul Raj                                                                                                Platinum Package
                    50000
```

3. Display services booked for each event

```
SQL> SELECT b.booking_id,s.service_name,bs.quantity,bs.unit_price
  2  FROM booking_service bs JOIN booking b ON bs.booking_id=b.booking_id
  3  JOIN service s ON bs.service_id=s.service_id;

BOOKING_ID SERVICE_NAME                                                                      QUANTITY UNIT_PRICE
---------- ---------------------------------------------------------------------------------------- ---------- ----------
         1 Decoration                                                                              1       5000
         1 Catering                                                                              200        250
         1 Photography                                                                             1       8000
         2 Decoration                                                                              1       5000
         2 Sound System                                                                            1       3000
         3 Catering                                                                              400        250
         3 Photography                                                                             1       8000
         4 Sound System                                                                            1       3000
         4 Lighting                                                                                1       4000
         5 Decoration                                                                              1       5000
```

4. Display payment details with customer names

```
SQL> SELECT p.payment_id,c.full_name,p.amount,p.payment_method
  2  FROM payment p JOIN booking b ON p.booking_id=b.booking_id
  3  JOIN customer c ON b.customer_id=c.customer_id;

PAYMENT_ID FULL_NAME                                                                                                 AMOUNT PAYMENT_METHOD
---------- ---------------------------------------------------------------------------------------------------------- ---------- ------------------
         1 Arun Kumar                                                                                                  30000 UPI
         2 Arun Kumar                                                                                                  15000 CARD
         3 Priya Sharma                                                                                                 5000 CASH
         4 Rahul Raj                                                                                                   40000 BANK_TRANSFER
```

2.6 Subqueries

1. Display customers who have made bookings

```
SQL> SELECT full_name FROM customer
  2  WHERE customer_id IN (SELECT customer_id FROM booking);

FULL_NAME
--------------------------------------------------------------------------------
Arun Kumar
Priya Sharma
Rahul Raj
Divya S
Karthik M
```

2. Display packages costing more than the average package price

```
SQL> SELECT package_name,base_price FROM hall_package
  2  WHERE base_price>(SELECT AVG(base_price) FROM hall_package);

PACKAGE_NAME                                                BASE_PRICE
---------------------------------------------------------- ----------
Diamond Package                                                 75000
Premium Package                                                100000
```

3. Display bookings with payments greater than ₹10,000

```
SQL> SELECT * FROM booking WHERE booking_id IN
  2  (SELECT booking_id FROM payment WHERE amount>10000);

BOOKING_ID CUSTOMER_ID EVENT_TYPE_ID PACKAGE_ID START_AT  END_AT    GUEST_COUNT STATUS       NOTES
---------- ----------- ------------- ---------- --------- --------- ----------- ------------ ---------------
----------------------------------------
         1           1             1          2 10-NOV-26 10-NOV-26         200 CONFIRMED    Wedding event
         3           3             3          3 15-NOV-26 15-NOV-26         400 CONFIRMED    Reception
```

2.7 Aggregate Functions

1. Count the total number of customers

```
SQL> SELECT COUNT(*) AS total_customers
  2  FROM customer;

TOTAL_CUSTOMERS
---------------
              5
```

2. Calculate the average hall package price

```
SQL> SELECT AVG(base_price) AS average_price
  2  FROM hall_package;

AVERAGE_PRICE
-------------
        54000
```

3. Calculate total payments received

```
SQL> SELECT SUM(amount) AS total_payments
  2  FROM payment;

TOTAL_PAYMENTS
--------------
         90000
```

4. Count bookings for each status

```
SQL> SELECT status,COUNT(*) AS total_bookings
  2  FROM booking GROUP BY status;

STATUS       TOTAL_BOOKINGS
------------ --------------
CONFIRMED                 3
COMPLETED                 1
CANCELLED                 1
```

5. Calculate total revenue collected for each booking

```
SQL> SELECT booking_id,SUM(amount) AS total_paid
  2  FROM payment GROUP BY booking_id;

BOOKING_ID TOTAL_PAID
---------- ----------
         1      45000
         2       5000
         3      40000
```

2.8 Views

1. Create a view displaying booking details

```
SQL> CREATE OR REPLACE VIEW booking_details AS
  2  SELECT b.booking_id,c.full_name,e.event_name,h.package_name,b.start_at,b.end_at,b.status
  3  FROM booking b JOIN customer c ON b.customer_id=c.customer_id
  4  JOIN event_type e ON b.event_type_id=e.event_type_id JOIN hall_package h ON b.package_id=h.package_id;

View created.
```

2. Display the booking detail's view

```
SQL> SELECT * FROM booking_details;

BOOKING_ID FULL_NAME       EVENT_NAME          PACKAGE_NAME             START_AT            END_AT    STATUS
---------- --------------- ------------------- ------------------------ ------------------- --------- ------------
         2 Priya Sharma    Birthday            Silver Package           12-NOV-26           12-NOV-26 CONFIRMED
         5 Karthik M       Engagement          Silver Package           20-NOV-26           20-NOV-26 CANCELLED
         1 Arun Kumar      Wedding             Gold Package             10-NOV-26           10-NOV-26 CONFIRMED
         4 Divya S         Corporate Event     Gold Package             18-NOV-26           18-NOV-26 COMPLETED
         3 Rahul Raj       Reception           Platinum Package         15-NOV-26           15-NOV-26 CONFIRMED
```

3. Create a view displaying total payments per booking

```
SQL> CREATE OR REPLACE VIEW booking_payment_summary AS
  2  SELECT booking_id,SUM(amount) AS total_paid
  3  FROM payment GROUP BY booking_id;

View created.
```

4. Display the payment summary view

```
SQL> SELECT * FROM booking_payment_summary;

BOOKING_ID TOTAL_PAID
---------- ----------
         1      45000
         2       5000
         3      40000
```

2.9 Additional useful management queries

1. Display confirmed bookings

```
SQL> SELECT booking_id,start_at,end_at,guest_count
  2  FROM booking WHERE status='CONFIRMED';

BOOKING_ID START_AT           END_AT    GUEST_COUNT
---------- ------------------ --------- -----------
         1 10-NOV-26          10-NOV-26         200
         2 12-NOV-26          12-NOV-26          80
         3 15-NOV-26          15-NOV-26         400
```

2. Find halls/packages that can accommodate 200 or more guests

```
SQL> SELECT package_name,capacity,base_price
  2  FROM hall_package WHERE capacity>=200;

PACKAGE_NAME                CAPACITY BASE_PRICE
--------------------------- ---------- ----------
Gold Package                       250      30000
Platinum Package                   500      50000
Diamond Package                    750      75000
Premium Package                   1000     100000
```

3. Display bookings with more than 100 guests

```
SQL> SELECT booking_id,guest_count,status FROM booking
  2  WHERE guest_count>100 ORDER BY guest_count DESC;

BOOKING_ID GUEST_COUNT STATUS
---------- ----------- ------------
         3         400 CONFIRMED
         1         200 CONFIRMED
         4         150 COMPLETED
```

4. Find customers who have not made any bookings

```
SQL> SELECT c.customer_id,c.full_name FROM customer c
  2  WHERE NOT EXISTS (SELECT 1 FROM booking b WHERE b.customer_id=c.customer_id);

no rows selected
```

Phase 3
PL/SQL Implementation and Testing

3.1 Stored Procedure - Create a Booking

```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> CREATE OR REPLACE PROCEDURE create_booking(
  2  p_customer IN NUMBER,p_event IN NUMBER,p_package IN NUMBER,
  3  p_start IN DATE,p_end IN DATE,p_guests IN NUMBER) AS
  4  BEGIN
  5  IF p_start>=p_end OR p_guests<=0 THEN
  6  RAISE_APPLICATION_ERROR(-20001,'Invalid booking details');
  7  END IF;
  8  INSERT INTO booking(customer_id,event_type_id,package_id,start_at,end_at,guest_count,status)
  9  VALUES(p_customer,p_event,p_package,p_start,p_end,p_guests,'PENDING');
 10  DBMS_OUTPUT.PUT_LINE('Booking created successfully');
 11  EXCEPTION
 12  WHEN OTHERS THEN
 13  DBMS_OUTPUT.PUT_LINE('Error: '||SQLERRM);
 14  END;
 15  /

Procedure created.

SQL> BEGIN
  2  create_booking(1,1,1,TO_DATE('2026-12-01 09:00','YYYY-MM-DD HH24:MI'),TO_DATE('2026-12-01 17:00','YYYY-MM-DD HH24:MI'),80);
  3  END;
  4  /
Booking created successfully

PL/SQL procedure successfully completed.
```

3.2 Function - Calculate Total Payment

```
SQL> CREATE OR REPLACE FUNCTION total_payment(p_booking_id NUMBER)
  2  RETURN NUMBER IS
  3  v_total NUMBER;
  4  BEGIN
  5  SELECT NVL(SUM(amount),0) INTO v_total
  6  FROM payment WHERE booking_id=p_booking_id;
  7  RETURN v_total;
  8  END;
  9  /

Function created.

SQL> SELECT booking_id,total_payment(booking_id) AS total_paid
  2  FROM booking ORDER BY booking_id;

BOOKING_ID TOTAL_PAID
---------- ----------
         1      45000
         2       5000
         3      40000
         4          0
         5          0
         6          0

6 rows selected.
```

3.3 Cursor - Display All Booking Details

```
SQL> CREATE OR REPLACE PROCEDURE display_bookings AS
  2  CURSOR c_bookings IS
  3  SELECT b.booking_id,c.full_name,e.event_name,b.status
  4  FROM booking b JOIN customer c ON b.customer_id=c.customer_id
  5  JOIN event_type e ON b.event_type_id=e.event_type_id;
  6  BEGIN
  7  FOR r IN c_bookings LOOP
  8  DBMS_OUTPUT.PUT_LINE(r.booking_id||' '||r.full_name||' '||r.event_name||' '||r.status);
  9  END LOOP;
 10  END;
 11  /

Procedure created.

SQL> BEGIN
  2  display_bookings;
  3  END;
  4  /
1 Arun Kumar Wedding CONFIRMED
6 Arun Kumar Wedding PENDING
2 Priya Sharma Birthday CONFIRMED
3 Rahul Raj Reception CONFIRMED
4 Divya S Corporate Event COMPLETED
5 Karthik M Engagement CANCELLED

PL/SQL procedure successfully completed.
```

3.4 Trigger - Check Hall Package Capacity

```
SQL> CREATE OR REPLACE TRIGGER check_package_capacity
  2  BEFORE INSERT OR UPDATE OF package_id,guest_count ON booking
  3  FOR EACH ROW
  4  DECLARE
  5  v_capacity NUMBER;
  6  BEGIN
  7  SELECT capacity INTO v_capacity FROM hall_package
  8  WHERE package_id=:NEW.package_id;
  9  IF :NEW.guest_count>v_capacity THEN
 10  RAISE_APPLICATION_ERROR(-20002,'Guest count exceeds package capacity');
 11  END IF;
 12  END;
 13  /

Trigger created.

SQL> UPDATE booking SET guest_count=50
  2  WHERE booking_id=1;

1 row updated.

SQL> UPDATE booking SET guest_count=1000
  2  WHERE booking_id=1;
UPDATE booking SET guest_count=1000
       *
ERROR at line 1:
ORA-20002: Guest count exceeds package capacity
ORA-06512: at "SYSTEM.CHECK_PACKAGE_CAPACITY", line 7
ORA-04088: error during execution of trigger 'SYSTEM.CHECK_PACKAGE_CAPACITY'
```

3.5 Exception Handling - Demonstration

```
SQL> DECLARE
  2  v_status booking.status%TYPE;
  3  BEGIN
  4  SELECT status INTO v_status FROM booking WHERE booking_id=9999;
  5  DBMS_OUTPUT.PUT_LINE('Status: '||v_status);
  6  EXCEPTION
  7  WHEN NO_DATA_FOUND THEN
  8  DBMS_OUTPUT.PUT_LINE('Error: Booking not found');
  9  WHEN TOO_MANY_ROWS THEN
 10  DBMS_OUTPUT.PUT_LINE('Error: Multiple bookings found');
 11  WHEN OTHERS THEN
 12  DBMS_OUTPUT.PUT_LINE('Unexpected error: '||SQLERRM);
 13  END;
 14  /
Error: Booking not found

PL/SQL procedure successfully completed.
```

3.6 Testing the Management System

1. Test case TC01 - Create a valid booking

```
SQL> BEGIN
  2  create_booking(1,1,1,TO_DATE('2026-12-01 09:00','YYYY-MM-DD HH24:MI'),TO_DATE('2026-12-01 17:00','YYYY-MM-DD HH24:MI'),80);
  3  END;
  4  /
Booking created successfully

PL/SQL procedure successfully completed.

SQL> SELECT * FROM booking
  2  ORDER BY booking_id;

BOOKING_ID CUSTOMER_ID EVENT_TYPE_ID PACKAGE_ID START_AT            END_AT    GUEST_COUNT STATUS       NOTES
---------- ----------- ------------- ---------- -------------------- --------- ----------- ------------ --------------------------
-------------------------------------------
         1           1             1          2 10-NOV-26            10-NOV-26          50 CONFIRMED    Wedding event
         2           2             2          1 12-NOV-26            12-NOV-26          80 CONFIRMED    Birthday party
         3           3             3          3 15-NOV-26            15-NOV-26         400 CONFIRMED    Reception
         4           4             4          2 18-NOV-26            18-NOV-26         150 COMPLETED    Annual corporate meeting
         5           5             5          1 20-NOV-26            20-NOV-26          60 CANCELLED    Engagement ceremony
         6           1             1          1 01-DEC-26            01-DEC-26          80 PENDING
         7           1             1          1 01-DEC-26            01-DEC-26          80 PENDING

7 rows selected.
```

2. Test case TC02 - End time earlier than start time

```
SQL> BEGIN
  2  create_booking(1,1,1,TO_DATE('2026-12-02 17:00','YYYY-MM-DD HH24:MI'),TO_DATE('2026-12-02 09:00','YYYY-MM-DD HH24:MI'),50);
  3  END;
  4  /
Error: ORA-20001: Invalid booking details

PL/SQL procedure successfully completed.
```

3. Test case TC03 - Calculate the total payment

```
SQL> SELECT booking_id,total_payment(booking_id) AS total_paid
  2  FROM booking ORDER BY booking_id;

BOOKING_ID TOTAL_PAID
---------- ----------
         1      45000
         2       5000
         3      40000
         4          0
         5          0
         6          0
         7          0

7 rows selected.

SQL> SELECT total_payment(1) AS total_paid
  2  FROM dual;

TOTAL_PAID
----------
     45000
```

4. Test case TC04 - Display all bookings

```
SQL> BEGIN
  2  display_bookings;
  3  END;
  4  /
1 Arun Kumar Wedding CONFIRMED
6 Arun Kumar Wedding PENDING
7 Arun Kumar Wedding PENDING
2 Priya Sharma Birthday CONFIRMED
3 Rahul Raj Reception CONFIRMED
4 Divya S Corporate Event COMPLETED
5 Karthik M Engagement CANCELLED

PL/SQL procedure successfully completed.
```

5. Test case TC05 - Try exceeding package capacity

```
SQL> UPDATE booking SET guest_count=1000
  2  WHERE booking_id=1;
UPDATE booking SET guest_count=1000
*
ERROR at line 1:
ORA-20002: Guest count exceeds package capacity
ORA-06512: at "SYSTEM.CHECK_PACKAGE_CAPACITY", line 7
ORA-04088: error during execution of trigger 'SYSTEM.CHECK_PACKAGE_CAPACITY'


SQL> SELECT booking_id,guest_count
  2  FROM booking WHERE booking_id=1;

BOOKING_ID GUEST_COUNT
---------- -----------
         1          50
```

6. Test case TC06 - Search for a nonexistent booking

```
SQL> DECLARE
  2  v_status booking.status%TYPE;
  3  BEGIN
  4  SELECT status INTO v_status FROM booking WHERE booking_id=9999;
  5  DBMS_OUTPUT.PUT_LINE('Status: '||v_status);
  6  EXCEPTION
  7  WHEN NO_DATA_FOUND THEN
  8  DBMS_OUTPUT.PUT_LINE('Error: Booking not found');
  9  END;
 10  /
Error: Booking not found

PL/SQL procedure successfully completed.
```

3.7 Verify the final database state

Check bookings:

```
SQL> SELECT booking_id,customer_id,package_id,guest_count,status
  2  FROM booking ORDER BY booking_id;

BOOKING_ID CUSTOMER_ID PACKAGE_ID GUEST_COUNT STATUS
---------- ----------- ---------- ----------- ------------
         1           1          2          50 CONFIRMED
         2           2          1          80 CONFIRMED
         3           3          3         400 CONFIRMED
         4           4          2         150 COMPLETED
         5           5          1          60 CANCELLED
         6           1          1          80 PENDING
         7           1          1          80 PENDING

7 rows selected.
```

Check total payments:

```
SQL> SELECT booking_id,SUM(amount) AS total_paid
  2  FROM payment GROUP BY booking_id;

BOOKING_ID TOTAL_PAID
---------- ----------
         1      45000
         2       5000
         3      40000
```

Check whether your PL/SQL objects compiled successfully:

```
SQL> SELECT object_name,object_type,status FROM user_objects
  2  WHERE object_type IN ('PROCEDURE','FUNCTION','TRIGGER')
  3  ORDER BY object_type,object_name;

OBJECT_NAME                                                                                                                                                      OBJECT_TYPE          STA
TUS
-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------- ------------------- ---
---------
TOTAL_PAYMENT                                                                                                                                                    FUNCTION             VAL
ID
CREATE_BOOKING                                                                                                                                                   PROCEDURE            VAL
ID
DISPLAY_BOOKINGS                                                                                                                                                 PROCEDURE            VAL
ID
CHECK_PACKAGE_CAPACITY                                                                                                                                           TRIGGER              VAL
ID
```